

Software Testing

SWEN1 | BIF3

Table of Contents

- Introduction into Software Testing
- Inspections and Testing
- Testing Intro and Unit Tests
- Test-Driven Development (TDD)



Introduction into Software Testing

General & Program Testing

Testing is intended to show that:

- a program does **what it is intended to do** and
- to **discover** program **defects** before it is put into use.

When you test software, you

- execute a program
- using **artificial data** and
- **check the results** of the test run for:
  - *errors* (“outcome which is not expected”),
  - *anomalies* (or defect, or bug, e.g. “unexpected behavior in terms of UX”) or
  - information about the program’s *non-functional attributes*.

Can reveal the presence of errors NOT their absence!

Goals

To **demonstrate** to the developer and the customer that the software **meets its requirements**.

- For custom software, this means that there should be at least **one test for each requirement** in the requirements document.
- For generic software products, it means that there should be **tests for all of the system features**, plus combinations of these features, that will be incorporated in the product release.

To **discover situations** in which the behavior of the software is incorrect, undesirable or **does not conform to its specification**.

- **Defect testing** is concerned with rooting out undesirable system behavior such as system crashes, unwanted interactions with other systems, incorrect computations and data corruption.

Validation- and Defect-Testing

The first goal leads to **validation testing**

- You expect the system to perform correctly using a given set of test cases that **reflect the system's expected use**.

The second goal leads to **defect testing**

- The test cases are designed to expose defects.
- The test cases in defect testing can be deliberately obscure and **need not reflect how the system is normally used**.

Test Process Goals

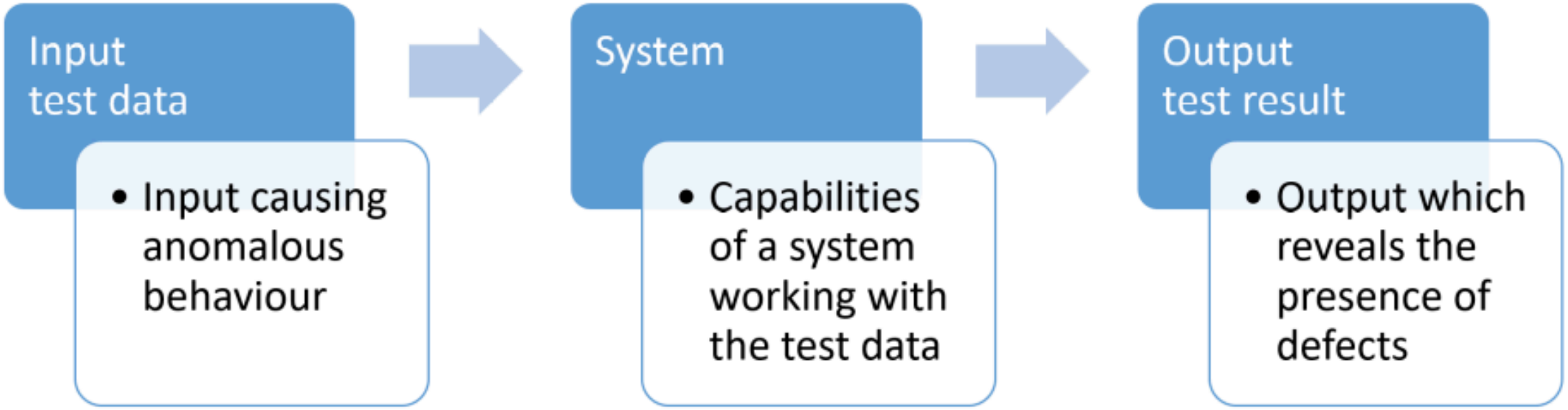
Validation testing

- To demonstrate to the developer and the system customer that the software meets its requirements
- A successful test shows that the system **operates as intended**.

Defect testing

- To discover faults or defects in the software where its behavior is incorrect or not in conformance with its specification
- A successful test is a test that makes the **system perform incorrectly and so exposes a defect** in the system.

Input-Output-Model



Verification vs Validation

- Verification: "**Are we building the product right**".
  The software should conform to its specification.
- Validation: "**Are we building the right product**".
  The software should do what the user really requires.

Aim of Validation and Verification is to establish **confidence that the system is 'fit for purpose'** or ready to use.

- Software purpose
  The level of confidence depends on how critical the software is to an organization.
- User expectations
  Users may have high/low expectations of certain kinds of software.
- Marketing environment
  Getting a product to market early may be more important than finding all defects.

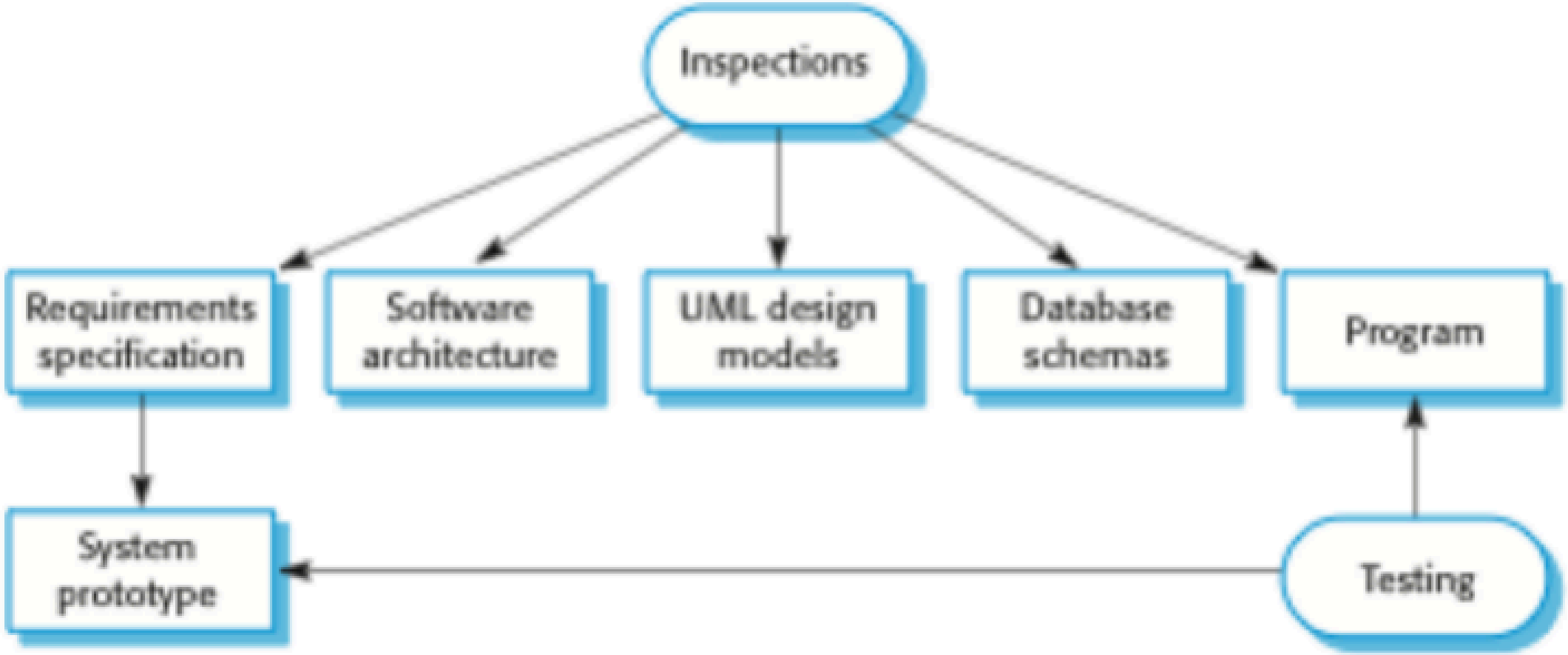


Inspections and Testing

Inspections and Testing

- Software inspections
  Concerned with analysis of the static system representation to discover problems (**static verification**)
  - May be supplement by tool-based document and code analysis.
- Software testing
  Concerned with exercising and observing product behavior (**dynamic verification**)
  - The system is executed with data and its operational behavior is observed.

Inspections and Testing in SLC



Software Inspection

- These involve people **examining the source** representation with the aim of discovering anomalies and defects.
- Inspections **not require execution** of a system so may be used before implementation.
- They **may be applied to any representation of the system** (requirements, design, configuration data, test data, etc.).
- They have been shown to be an **effective technique for discovering program errors**.

Advantages of inspections

- During testing, errors can mask (hide) other errors. Because inspection is a static process, you **don’t have to be concerned with interactions between errors (don’t mask each other)**.
- **Incomplete versions** of a system **can be inspected** without additional costs. If a program is incomplete, then you need to develop specialized test harnesses to test the parts that are available.
- As well as searching for program defects, an inspection **can also consider broader quality attributes (metrics)** of a program, such as compliance with standards, portability and maintainability.

Inspections and Testing

- Inspections and testing are complementary and not opposing verification techniques.
- Both should be used during the V & V process.
- Inspections can check conformance with a specification.
- Inspections can’t check conformance with customer’s requirements.
- Inspections can’t check non-functional characteristics such as performance, usability, etc.

A Model of the software testing process





Testing Intro and Unit Tests

Stages of Testing

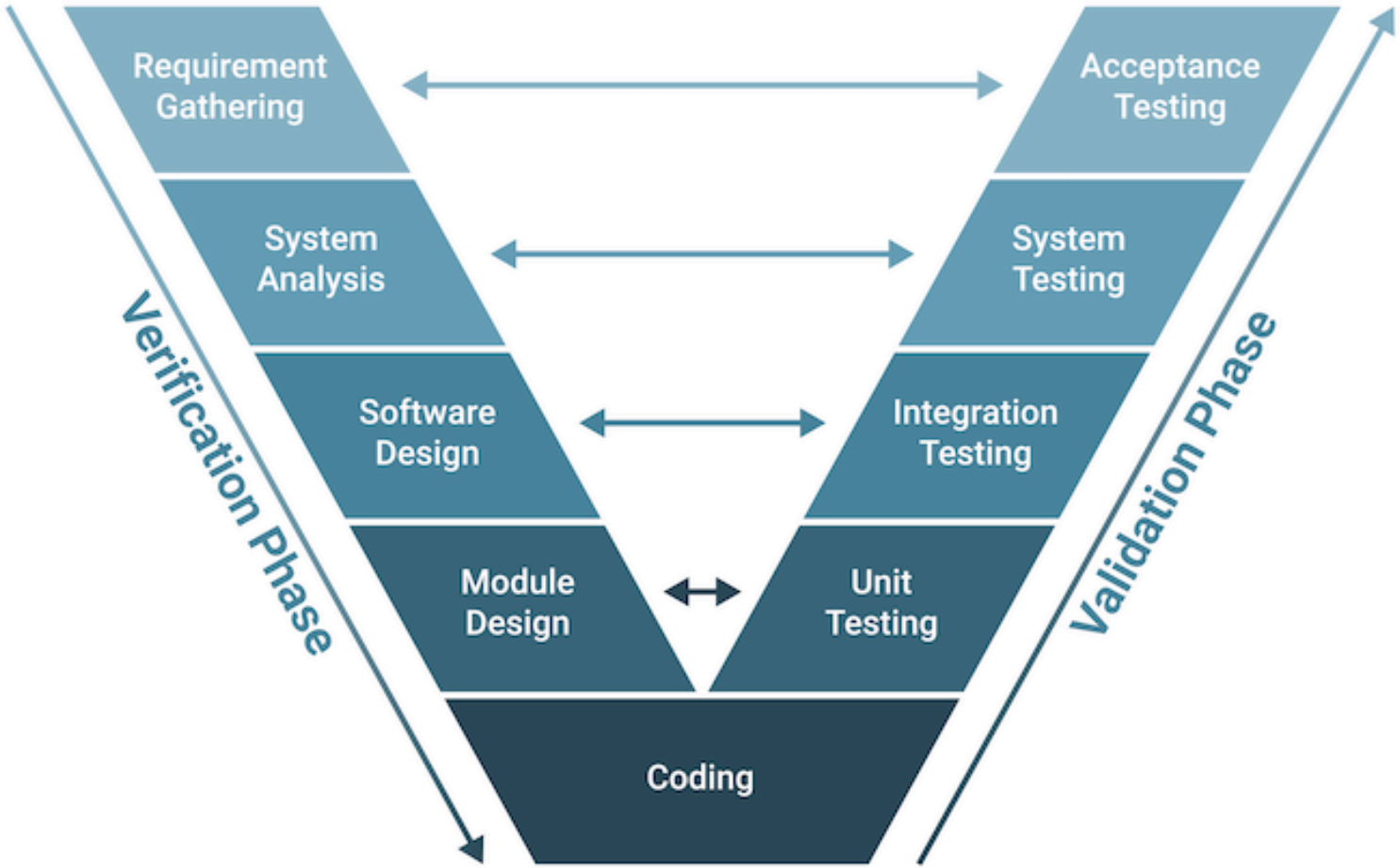
- Development testing:
  where the system is tested during development to discover bugs and defects.
- Release testing:
  where a separate testing team test a complete version of the system before it is released to users.
- User testing:
  where users or potential users of a system test the system in their own environment.

Development Testing

Development testing includes **all testing activities** that are carried out **by the team developing the system**.

- **Unit testing**, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
- **Component testing**, where **several individual units** are integrated to create composite components. Component testing should focus on testing component interfaces.
- **System testing**, where some or all the components in a system are integrated and the system is tested **as a whole**. System testing should focus on testing component interactions.

V-Model



see https://builtin.com/software-engineering-perspectives/v-model

Unit Testing

- Unit testing is the process of **testing individual components in isolation**.
- It is a **defect testing** process.
- Units may be:
  - Individual **methods** within an object
  - **Classes** with several attributes and methods
  - Composite **components** with defined interfaces used to access their functionality.

Class Testing

- Complete test coverage of a class involves
  - Testing all operations associated with an object
  - Setting and getting all object attributes
  - Exercising the object in all possible states.
- Inheritance makes it harder to design object class tests as the information to be tested is not locally.

Automated Testing

- Whenever possible, unit testing **should be automated** so that tests are run and checked without manual intervention.
- In automated unit testing, you make use of a **test automation framework** (such as NUnit/JUnit) to write and run your program tests.
- Unit testing frameworks provide generic test classes that you extend (or some kind of markup) to create specific test cases. They can then **run all of the tests** that you have implemented and report, often through some GUI.

Automated Test Components - AAA

- A **setup** part, where you initialize the system with the test case, namely the inputs and expected outputs.
- A **call** part, where you call the object or method to be tested.
- An **assertion** part where you compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful if false, then it has failed.

The **AAA (Arrange – Act - Assert)** pattern is a common way of writing unit tests for a method under test.

- **Arrange**: initialize objects and set the value of the data that is passed to the method under test.
- **Act**: invoke the method under test with the arranged parameters.
- **Assert**: verify that the action of the method under test behaves as expected.

https://learn.microsoft.com/en-us/visualstudio/test/unit-test-basics?view=vs-2022

Unit Test Types

- The first of these should reflect normal operation of a program and should show that the component **works as expected**.
- The other kind of test case should be based on testing experience of where common problems arise. It should use **abnormal inputs** to check that these are properly processed and **do not crash** the component.

Design Approaches

- **Blackbox Testing** - Testing only based on the requirements, without further knowledge of the details of the software internals and it's source code.
- **Whitebox Testing** – Test-Scenarios and Test-Cases based on implementation details
- **Graybox Testing** - see https://www.tutorialspoint.com/differences-between-white-box-testing-and-grey-box-testing
- **Bottom-Up Testing** - first lower hierarchy is tested individually and then the components that rely upon these
- **Top-Down Integration Testing** - integration testing technique especially used to test behavior with mocked components

Test creation methods

...to design effective test cases.

Equivalence Partitioning and Boundary Value Analysis are two popular **methods used in unit test creation**:

1. **Equivalence Partitioning** focuses on dividing input data into different equivalence classes to design effective test cases.
2. **Boundary Value Analysis** targets the boundaries or limits of input values to identify potential errors or issues.

Equivalence Partitioning

Divide input data into different equivalence classes

1. Identify the input conditions that affect the behavior of the software.
2. Divide the input data into equivalence classes based on these conditions.
3. Each equivalence class represents a unique set of test cases.
4. Select representative test cases from each equivalence class to ensure maximum coverage.

This method helps to reduce the number of test cases while still providing adequate coverage of the input space.

Boundary Value Analysis

Targets the boundaries or limits of input values to identify potential errors or issues.

1. Identify the input variables and their boundaries.
2. Select test cases that represent the minimum and maximum values, as well as values just above and below the boundaries.
3. Test cases are designed to target the boundaries and potential edge cases.
4. This method helps to identify errors or issues that may occur at the boundaries of acceptable input ranges.

It provides more thorough testing and increases the likelihood of detecting faults.

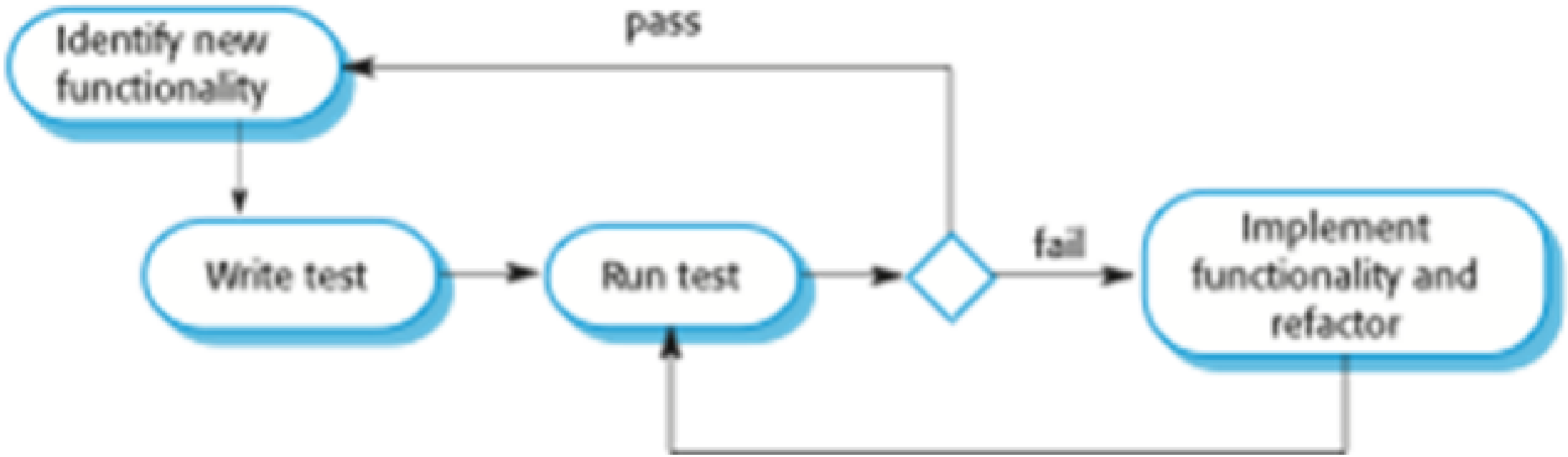


Test-Driven Development (TDD)

Test Driven Development

- Test-driven development (TDD) is an approach to program development in which you **combine testing- and code development**. (Kent Beck 2003)
- **Tests are written before code** and ‘passing’ the tests is the critical driver of development. (see: Red/Green/Refactor)
- You develop code incrementally, along with a test for that increment. **You don’t move on to the next increment until the code that you have developed passes its test**.
- TDD was introduced as **part of agile methods** such as Extreme Programming.
  However, it can also be used in plan-driven development processes.

TDD Process & Benefits



- **Code coverage**: Every code segment that you write has at least one associated test, so all code written has at least one test.
- **Regression testing**: A regression test suite is developed incrementally.
- Finding defects: When tests fail, it should be easier to find its source.
- System documentation: The tests themselves are a form of documentation that describe what the code should be doing.

Regression Testing

- Regression testing is testing the system to check that **changes have not 'broken'** previously working code.
- In a **manual** testing process, regression testing is **expensive** but, with automated testing, it is simple and straightforward. All tests are rerun every time a change is made to the program.
- Tests must run 'successfully' before the change is committed.

Further list of testing techniques

- **Single-Tests** (Checking single functions/methods or scenarios)
- **Regression-Tests** (Repeating of all executed tests, to check if everything is working as expected after changes where applied to parts or the whole software system)
- **Developer-Tests** (structured approach to quality-related activities performed by developers)
- **Load-Tests** (determines a system's performance under a specific expected load)
- **Performance Tests** (determines a system's performance without any pressure on the system)
- **Acceptance-Tests** (conducted to determine if the requirements of a specification or contract are met)

  ...

  see https://www.tutorialspoint.com/software_testing_dictionary/